Agentic AI for Public Administration: A Multi-Agent Architecture and Controlled Fault Injection Evaluation

Radu-Mihai Gheorghe[1,2] Petru-Liviu Bouruc[2] Ciprian Paduraru[2] Alin Stefanescu[2,3]

[1]UiPath, Romania

[2]University of Bucharest, Romania

[3]Institute for Logic and Data Science, Romania

30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026)

Artifact: `github.com/radugheo/agentic-public-administration`

PART 1

Motivation

The fragmentation problem

Romania's fiscal services are spread over three national portals, each with its own interface, vocabulary and procedural logic:

- **SPV**, *Spațiul Privat Virtual* (Virtual Private Space): filing declarations, consulting your fiscal record
- **Ghișeul.ro**, from *ghișeu* (service counter): paying taxes and contributions
- **RO e-Factura**: national electronic invoicing

In one year a freelancer declares annual income, pays pension and health contributions, registers a rental contract and issues an invoice, across portals sharing no state.

Where the burden sits

Computing the amounts is simple arithmetic. The effort goes into **which declaration applies, under which deadline, on which portal**, and the rules that settle each.

A concrete request

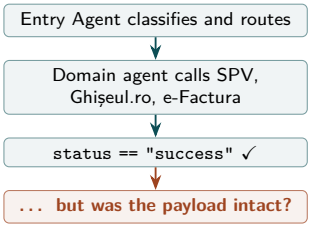
The task

"I earned 150,000 RON as a freelancer last year. What do I owe?"

Three amounts, under the 2026 rules, on a minimum gross salary of 4,050 RON:

CAS pension	25% of 24 × 4,050	24,300
CASS health	10% of 150,000	15,000
Impozit tax	10% of 110,700	11,070
Total		**50,370**

What the citizen never sees



What this paper does

Build

A multi-agent assistant for Romanian tax and fiscal administration. An Entry Agent classifies the citizen's request and routes it to one of five domain agents, over four shared services, built with UiPath Coded Agents on LangGraph: D212 filing, property sale, rental income, fiscal certificates and RO e-Factura.

Break

Every call the assistant makes into SPV, Ghișeul.ro and e-Factura is instrumented with four chaos operators: timeout, delay, partial response and corruption, at a configurable rate, replayable from a seed.

Measure

For each injected fault, look at the answer the citizen actually receives. Did the failure surface, or did it pass through silently?

Research questions

RQ1 How much gets through?

When a service behind the assistant is slow, returns an incomplete payload or a corrupted one, **what fraction of those faults reaches the citizen with no signal at all**, in an answer that reads as a normal success?

RQ2 Is that rate incidental, or structural?

Is the fraction a property of this implementation, to be driven down by better engineering, or is it **fixed by the shape of the response schema and the predicate the agent branches on**, and therefore predictable before the system is ever run?

PART 2

The system

How the system is organised

The Entry Agent

Every request enters here. The agent classifies it into intent, confidence, reasoning and extracted entities. A fixed table maps intent to a graph node, and low confidence diverts to clarification.

Four shared services

Document intake (OCR and schema validation), RAG over the Romanian Tax Code, deterministic calculation in decimal arithmetic, and payment through Ghișeul.ro.

Five domain agents

PFA/freelancer (D212), property sale, rental income, fiscal certificate and RO e-Factura. Each one reads the shared state, calls the shared services and writes the state back.

Infrastructure: UiPath Coded Agents

One environment for model access, retrieval indices, storage buckets and human review tasks, so none of it has to be assembled separately.

Hierarchical supervisor pattern, implemented as a LangGraph `StateGraph` with typed shared state and conditional routing.

What the UiPath platform provides

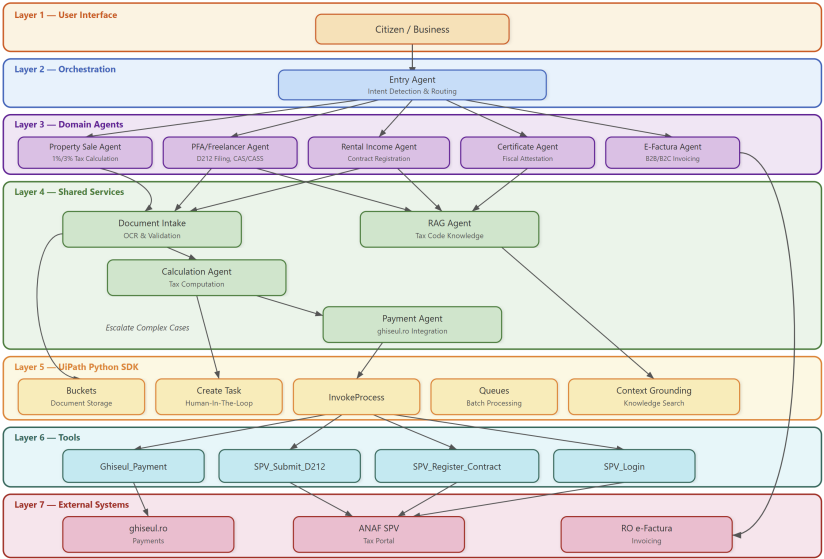
A bare LangGraph implementation still leaves model access, retrieval infrastructure, escalation and document storage to be assembled separately. Here they come from one execution environment.

Concern	UiPath resource
Model access	LLM Gateway, serving GPT-4o
Grounding in the Tax Code	Context Grounding indices, one per domain
Document storage	Storage buckets
Human escalation	Action Center tasks

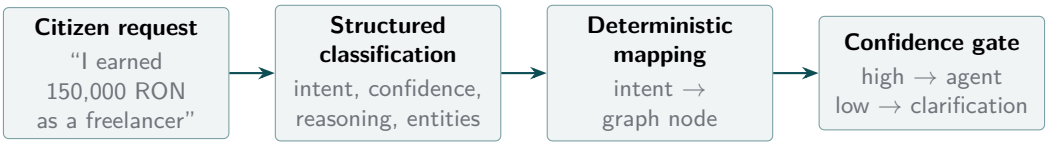
Built with **UiPath Coded Agents**, on the open-source UiPath LangChain SDK (`uipath-langchain`).

A LangGraph `StateGraph` declared through `uipath.json`, run and evaluated with the `uipath` CLI.

The architecture



Intent classification and routing



- The LLM classifies; it does not decide the route. A fixed table turns the predicted intent into a graph node, so the routing step itself can be inspected.
- Below the confidence threshold the request goes to clarification, even when an intent was assigned. Dispatching the wrong specialist here means **computing the wrong tax**.

The fault injector

Mechanism

A decorator wraps each simulated government boundary. **One Bernoulli draw per call** decides whether a fault fires; if it does, exactly one operator is applied, drawn uniformly, so each accounts for a quarter of injected faults. Every event is logged with the operation and the operator type.

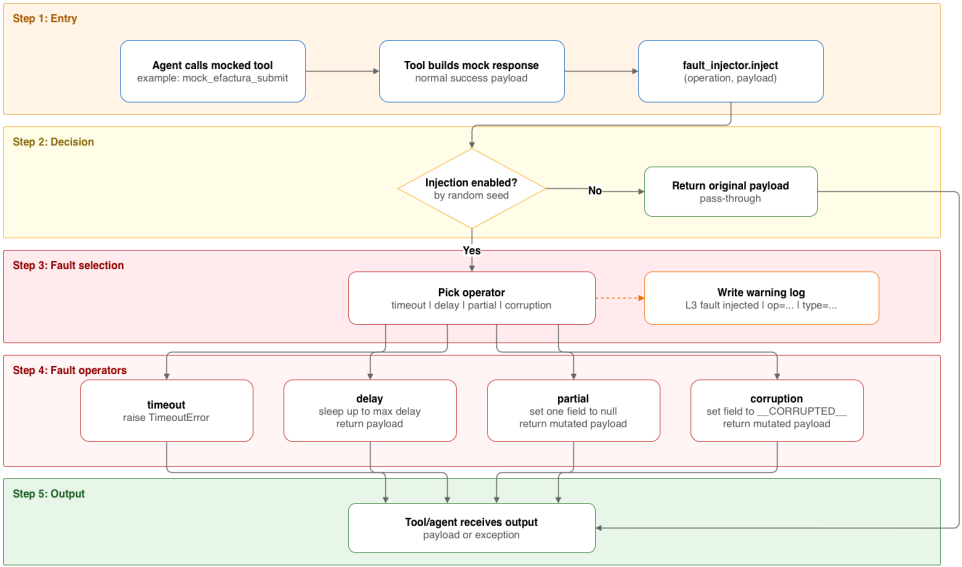
Parameters

s: seed, for deterministic replay
η: per-call injection rate
$\Delta t_{\max}$: bound on the induced delay

At $\eta = 0$ the nominal system is recovered unchanged.

Operator	Effect on the call
Timeout	raises `TimeoutError`, no payload
Delay	sleeps up to $\Delta t_{\max}$, payload intact
Partial	one field set to `None`
Corruption	one field replaced by a sentinel

Injection decision flow



Why the faults are silent

Agent control flow usually turns on a single field: a predicate such as `status == "success"`. The rest of the response object carries content the branch never inspects.

A single-field perturbation that misses `status` leaves the predicate true, so the agent takes the success branch anyway. For a response with $|\mathcal{F}|$ fields whose branch reads exactly one of them:

$$P(\text{silent}) = 1 - \frac{1}{|\mathcal{F}|}$$

The response above has four fields, so a hit misses `status` three times in four.

Expected silent rate

Operator	Share	Silent	Contributes
Timeout	0.25	0	0
Delay	0.25	1	0.2500
Partial	0.25	0.75	0.1875
Corruption	0.25	0.75	0.1875
Expected silent rate			**0.625**

Operators are drawn uniformly, so each is a quarter of injected faults. Delay preserves the payload, timeout always raises, and partial and corruption miss `status` with probability 0.75.

PART 3

Evaluation

Experimental setup

Prototype

Python 3.11, LangGraph 0.2, GPT-4o, running on the UiPath platform. Government calls run against a boundary layer that reproduces the portal schemas, so no live portal traffic is required.

Runs

$N = 100$ boundary calls, each an independent trial with at most one injected fault, sampled across the 8 portal-facing operations by call frequency in the five demo workflows.

$\eta \in \{0,\ 0.25,\ 0.50,\ 0.75,\ 1.00\}$, $\quad s = 17$, $\Delta t_{\max} = 250\,\text{ms}$.

Detectable

The response signals failure: an exception propagates, or an explicit error or degraded-service notice reaches the user.

Silent pass-through

Everything else counts as silent: the citizen-facing answer looks normal.

The unit of analysis is the call, not the session. A full workflow crosses several boundaries, each with its own draw.

A complete trace

A citizen reports PFA income of 150,000 RON and asks what contributions are owed.

1. Entry Agent classifies `pfa_d212_filing` with confidence 0.95
2. PFA Agent computes CAS and CASS and asks for confirmation
3. On the confirmation turn the SPV submission call is intercepted at $\eta = 1.0$, $s = 17$

```
[WARN] fault injected | op=mock_spv_submit_d212 | type=partial
```

The `message` field of `SPVSubmissionResult` was set to `None`. `status` stayed `"success"`, so the agent followed the success branch and answered:

What the citizen saw

"The D212 declaration was successfully submitted! ID: D212-01B66C85C3, Timestamp: 2026-03-27T22:20:32."

The response has four fields and the branch reads only one of them, so a fault that hits a single field goes unnoticed with probability 0.75. The same arithmetic applies to any response whose branch reads one field.

Fault injection results

η	Faults	TO	DL	PT	CR	Silent	Detectable
0.00	0	0	0	0	0	0 (n/a)	0 (n/a)
0.25	25	6	6	7	6	16 (64%)	9 (36%)
0.50	50	12	13	12	13	32 (64%)	18 (36%)
0.75	75	19	19	18	19	47 (63%)	28 (37%)
1.00	100	25	25	25	25	**63 (63%)**	37 (37%)

$N = 100$ calls, $s = 17$, $\Delta t_{\max} = 250$ ms. TO, DL, PT, CR: timeout, delay, partial, corruption.

- The measured rate matches the predicted **0.625**. Raising η raises the *count* of silent faults, but the proportion does not move.
- Delay leaves the payload intact, so it is silent by construction. It is also imperceptible: even at $\eta = 0.50$ it adds around 65 ms to a whole scenario.
- The same arithmetic carries to other agents: wherever a branch turns on one field, the exposure follows from the response schema, and **widening what the branch inspects is what lowers it**.

Cost feasibility

Per session
$0.05
one classification, two to three domain turns and one retrieval step

Small office
$51
per month, 50 sessions a day

Large agency
$5,060
per month, 5,000 sessions a day

- About 20,000 input and 2,100 output tokens per session, priced at $1.25 per million input and $10.00 per million output tokens,[†] over 22 working days a month. Licensing, infrastructure and maintenance are excluded.
- Cost scales linearly with volume and remains low at institutional scale. Routing with a smaller model and reserving the capable one for domain reasoning lowers it further.

At these volumes, cost is not what blocks deployment. Assurance is.

[†] Pricing is the GPT-5 public snapshot at the time of the experiment; the prototype itself runs GPT-4o. The figures indicate order of magnitude under an illustrative deployment scenario, not a fixed deployment cost.

PART 4

Conclusion

Conclusion

RQ1 How much gets through?

63% of injected boundary faults reach the citizen with no signal: 16 of 25 at $\eta = 0.25$, 63 of 100 at $\eta = 1.00$. Close to two faults in every three arrive dressed as an ordinary success message.

RQ2 Incidental, or structural?

Structural. The rate is set by the operator mix and the number of fields in the response object, matches $0.25 + 2 \times 0.25 \times 0.75 = 0.625$ in closed form, and does not move with the injection rate. The branch is already correct with respect to the flag it reads, so the fix belongs in an **output contract**, not in the branch.

A coherent interface is no evidence that the execution behind it was correct.

What remains to be tested

The 63% is a property of *this* architecture: a fixed routing graph whose branches read one status field. Each item below changes something that rate depends on, and none has been measured yet.

1. **Output contracts and guardrails.** Validity conditions on the assembled response before it reaches the citizen, and schema and policy checks at the boundary itself: a submission identifier that must be present and well formed, a payment code that must match its shape. Both catch what a branching predicate structurally cannot, and how much each recovers is an open number.
2. **Other agent architectures.** A **ReAct loop** choosing freely among the shared tools, or a **skill-driven agent** selecting a procedure at run time, reads different fields on different turns. Whether that raises detection or moves the blind spot is untested.
3. **Trace-based assurance.** Message-action traces, deterministic replay and action-level governance, so a silent condition stays recoverable after the fact.
4. **Realistic conditions.** Live portals in place of simulated ones, and faults compounding across a full session rather than one call at a time.

Thank you

Questions?

Radu-Mihai Gheorghe · Petru-Liviu Bouruc · Ciprian Paduraru · Alin Stefanescu

UiPath · University of Bucharest · Institute for Logic and Data Science

BACKUP SLIDES

Additional material

Backup: the five domain agents

PFA / Freelancer

Declarația Unică (D212). Computes the pension contribution (CAS, 25% on a band-stepped base) and the health contribution (CASS, 10% of net income, floor 6× and ceiling 72× the minimum gross salary). Submission through SPV.

Property sale

3% of the sale value under three years of ownership, 1% at three years or more. Payment guidance through Ghișeul.ro.

Rental income

Contract registration with ANAF and a 10% flat tax on annual rent.

Certificate

Certificat de atestare fiscală. It identifies the certificate type, checks the document requirements and guides submission.

RO e-Factura

B2B and B2C invoicing: invoice data collection, XML generation to the national standard, submission and status monitoring.

All five produce a standardised response field from the final model output, which keeps the formatting consistent across workflows.

Backup: the four shared services

Document intake

OCR plus validation against document-specific schemas for D212 forms, rental contracts and invoices. Returns structured data with confidence scores; below 80% the document is flagged for manual review.

RAG

Retrieval over dedicated indices of the Romanian Tax Code and procedural guidance, so every answer can be traced to a passage in the law and none of it rests on what the model happens to remember.

Calculation

All arithmetic is decimal, never floating point. Rates and thresholds (minimum salary, contribution percentages) are read from configuration, so annual regulatory updates need no code change.

Payment

Builds the payment request from the shared context, initiates it through Ghișeul.ro and returns the transaction details, including the redirect URL.

The division of labour is the same in all four: the model interprets the request, deterministic code performs the computation.

Backup: where the work sits

Agentic orchestration

LangGraph models control flow as a typed state machine. Each transition is explicit and can be audited. That matters when the service is regulated.

Retrieval

Tax law is specific and amended often. Retrieval grounds explanations in the Romanian Tax Code and is kept separate from the deterministic computation.

Chaos engineering

Faults are introduced deliberately and their effects observed. The technique reached LLM agents recently, though mostly in generic or synthetic testbeds.

Closest neighbours

Work on faults in agentic repositories identifies the boundary between probabilistic model output and deterministic program logic as a dominant failure class. That is the same interface targeted here. Reliability benchmarks for LLM agents include timeout, partial-response and schema-drift conditions, but do not connect the observed silent rate to schema structure in a deployed public-service setting.

Backup: cost model

Institution size	Sessions/day	Sessions/month	Cost/month
Small office	50	1,100	$51
Medium office	500	11,000	$506
Large agency	5,000	110,000	$5,060

22 working days per month; $1.25 per 1M input and $10.00 per 1M output tokens.

- Per session: one Entry Agent call (≈ 2,000 input / 200 output tokens), two to three domain turns (≈ 5,000 / 500 each), one retrieval-grounded step (≈ 3,000 / 400). About 20,000 input and 2,100 output tokens in total.
- Costs scale at roughly $1.01 per daily session per month.
- The prototype runs GPT-4o; the projection uses the pricing of a stronger model as an illustrative deployment scenario.

Backup: reproducibility

Deterministic replay

The injector draws from a seeded generator, so a fixed $(s, \eta, \Delta t_{\max})$ reproduces the same sequence of operators over the same sequence of boundary calls. Runs reported here use $s = 17$.

Fault log

Each injected event is written as a warning naming the operation and the operator, which is what makes the qualitative trace on the earlier slide checkable:

```
[WARN] fault injected | op=...  | type=...
```

Calibration

Delay sleeps for a draw from $\mathcal{U}[1, \Delta t_{\max}]$. Measured across affected calls: 125.5 ± 72.2 ms, against 125.5 and 71.9 predicted for $\mathcal{U}[1, 250]$, so the operator draws from the interval it claims.

Turning injection off

Setting $\eta = 0$ returns every payload unchanged, so the same code path serves both the nominal system and the perturbed one. The injector never modifies the portal stub logic itself.